

Simulador de uma plataforma de **cloud computing** — gerenciamento de máquinas virtuais, discos de armazenamento e monitoramento de recursos. Projeto desenvolvido para a disciplina de **Programação Orientada a Objetos 2 (POO2)**.

A aplicação permite que usuários se autentiquem, criem e gerenciem recursos de nuvem (VMs e discos), anexem/desanexem armazenamento, controlem o ciclo de vida das máquinas (iniciar/parar) e acompanhem métricas de uso (CPU, RAM) em tempo simulado.

Objetivo

Aplicar na prática os conceitos de POO (herança, polimorfismo, encapsulamento) em um sistema real com arquitetura em camadas, usando **Spring Boot** e persistência em banco relacional **PostgreSQL**.

O domínio gira em torno da classe abstrata `Recurso`, da qual herdam `VirtualMachine` e `Armazenamento` — mapeada via JPA com estratégia de herança `JOINED` (uma tabela-mãe `recurso` + tabelas filhas ligadas por chave estrangeira).

Stack

Item	Tecnologia
Linguagem	Java 21 (LTS)
Framework	Spring Boot 4.x
Build	Maven (<code>pom.xml</code>)
Banco de dados	PostgreSQL
ORM	Spring Data JPA (Hibernate)
Boilerplate	Lombok
Front-end	HTML + CSS + JavaScript (<code>fetch</code>) — servido estático pelo Spring

Arquitetura em camadas

```
Front (HTML/CSS/JS em resources/static/)
  ↓ HTTP (fetch)
Controller (@RestController – expõe a API REST)
  ↓ chama
Service (@Service – lógica de negócio, validação, permissão)
  ↓ usa
Repository (interface JpaRepository – persistência)
  ↓ persiste
Model (@Entity – entidades JPA)
```

Regra de ouro: nenhuma camada pula a vizinha — o Controller nunca chama o Repository direto, sempre passa pelo Service.

► Como rodar

Pré-requisitos

- **JDK 21**
- **Maven** (ou use o wrapper `./mvnw` incluído no projeto)
- **PostgreSQL** instalado e rodando

1. Crie o banco de dados

```
CREATE DATABASE wagacloud_dev;    -- desenvolvimento
CREATE DATABASE wagacloud;        -- definitivo (opcional)
```

2. Configure as credenciais

Os arquivos de configuração com senha **não vão para o Git**. Copie os exemplos e ajuste com a senha do seu PostgreSQL local:

```
cp src/main/resources/application-dev.properties.EXEMPLO src/main/resources/application-dev.properties
cp src/main/resources/application-prod.properties.EXEMPLO src/main/resources/application-prod.properties
```

Edite `application-dev.properties` e troque `SUA_SENHA_LOCAL` pela senha do seu Postgres:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/wagacloud_dev
spring.datasource.username=postgres
spring.datasource.password=SUA_SENHA_LOCAL
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

3. Suba a aplicação

```
./mvnw spring-boot:run
```

O perfil `dev` já é o ativo por padrão. Para usar o banco definitivo:

```
./mvnw spring-boot:run -Dspring-boot.run.profiles=prod
```

4. Acesse

Abra **http://localhost:8080** no navegador. Front e back rodam no mesmo servidor (sem CORS para resolver).

Ao subir, o `DataLoader` popula o banco automaticamente. Use uma das contas de teste:

Email	Senha	Nível
admin@cloud.com	123	ADMIN
maria@cloud.com	123	COMUM
joao@cloud.com	123	COMUM

☆☆ Funcionalidades

🔒 Autenticação e acesso

- Login e cadastro de usuários
- Dois níveis de acesso: **ADMIN** e **COMUM** (admin tem funcionalidades extras)

🖥 Máquinas Virtuais (VMs)

- Criar VM (CPU, RAM, sistema operacional) — já cria um disco padrão anexado
- Listar e deletar recursos do usuário
- **Iniciar** e **parar** VMs (controle de status)

💾 Armazenamento (Discos)

- Criar disco de armazenamento
- **Anexar** / **desanexar** disco de uma VM
- **Expandir** e **reduzir** capacidade do disco

📊 Monitoramento

- Geração de métricas simuladas (CPU 10–95%, RAM 20–80%)
- Listagem de métricas por recurso
- Listagem de recursos em estado **crítico** (destacados em vermelho)

API REST

Base: `http://localhost:8080`

Autenticação

Método	Endpoint	Descrição
POST	/api/login	Login (email + senha)
POST	/api/cadastro	Cadastra novo usuário

Recursos (VMs e Discos)

Método	Endpoint	Descrição
GET	/api/recursos	Lista recursos do usuário
POST	/api/vms	Cria VM (com disco padrão)
POST	/api/storages	Cria disco de armazenamento
PUT	/api/vms/{id}/iniciar	Inicia a VM
PUT	/api/vms/{id}/parar	Para a VM
PUT	/api/discos/{id}/expandir	Expande o disco
PUT	/api/discos/{id}/reduzir	Reduz o disco
PUT	/api/discos/anexar	Anexa disco a uma VM
PUT	/api/discos/desanexar	Desanexa disco
DELETE	/api/recursos/{id}	Deleta um recurso

Monitoramento

Método	Endpoint	Descrição
GET	/api/monitoramento/{recursoId}	Métricas de um recurso
POST	/api/monitoramento/gerar	Gera métricas simuladas
GET	/api/monitoramento/criticos	Lista recursos críticos

Nota sobre autenticação: o projeto não usa sessão/token. Após o login, o front guarda o `id` do usuário e o envia nos requests que precisam (ex: `?usuarioId=1`). É uma simplificação consciente, adequada ao escopo acadêmico — não seguro para produção.

Estrutura do projeto

```
src/main/java/com/WagaCloud/wagacloud/
├─ WagacloudApplication.java      ← classe main (@SpringBootApplication)
├─ model/                          ← entidades JPA
│   ├── enums/                    ← NivelAcesso, StatusRecurso, TipoDisco
│   ├── Usuario.java
│   ├── Recurso.java              ← abstrata (herança JOINED)
│   ├── VirtualMachine.java       ← extends Recurso
│   ├── Armazenamento.java       ← extends Recurso
│   └─ Monitoramento.java
├─ repository/                    ← interfaces JpaRepository
├─ service/                       ← lógica de negócio
├─ controller/                   ← endpoints REST
├─ config/
│   └─ DataLoader.java            ← popula o banco ao subir
src/main/resources/static/        ← front-end (HTML, CSS, JS)
```

Para detalhes de planejamento, fases e divisão de tarefas, veja [docs/ROADMAP.md](#).

👥 Equipe

Integrante	Feature
Bruck	Autenticação + classes-base + integração
Maju	Recursos (VMs e Armazenamento)
Marcelo	Monitoramento